

---

# RAPPORT DE PROJET

## Système de Gestion d'École

*Application Java avec MySQL - Modeles, JDBC et DAO*

|                       |                                     |
|-----------------------|-------------------------------------|
| Module :              | Programmation Orientée Objet / IHM  |
| Réalisé par :         | Sassi Mohamed Ayoub / Rekik Mohamed |
| Encadrante :          | Mme Akrouit Nesrine                 |
| Année universitaire : | 2025 / 2026                         |

# Table des matières

|                                                |  |
|------------------------------------------------|--|
| 1. Introduction .....                          |  |
| 2. Présentation du projet .....                |  |
| 3. Architecture du système .....               |  |
| 4. Conception UML .....                        |  |
| 4.1 Diagramme des cas d'utilisation .....      |  |
| 4.2 Diagramme de classes .....                 |  |
| 4.3 Diagramme de séquence .....                |  |
| 5. Base de données .....                       |  |
| 5.1 Structure des tables .....                 |  |
| 5.2 Schéma relationnel .....                   |  |
| 6. Structure du projet Java .....              |  |
| 7. Description des interfaces graphiques ..... |  |
| 8. Fonctionnalités par rôle .....              |  |
| 9. Conclusion .....                            |  |

# 1. Introduction

Dans le cadre du module de Programmation Orientee Objet et IHM, ce projet presente une application Java de gestion d'ecole basee sur l'architecture MVC, la connexion MySQL par JDBC et le patron DAO (Data Access Object). Le projet est structure autour des couches model, view, controller, dao, config et util.

Le systeme permet de gerer les utilisateurs de l'etablissement : etudiants, enseignants, personnel administratif et comptes d'authentification. Les interfaces graphiques Swing communiquent avec les controleurs, qui appellent les DAO pour executer les requetes SQL vers MySQL.

## 2. Présentation du projet

### 2.1 Objectifs

L'objectif principal de ce projet est de concevoir et développer une application de gestion scolaire permettant de :

- Gerer les informations des etudiants, enseignants, matieres, notes et utilisateurs
- Représenter l'heritage entre Utilisateur, Etudiant, Enseignant et Personnel
- Acceder aux tables MySQL a travers des classes DAO generiques et specialisees
- Organiser les interactions entre vues Swing, controleurs, DAO et base MySQL
- Assurer la persistance des donnees via MySQL

### 2.2 Technologies utilisées

| Technologie   | Role                                                  |
|---------------|-------------------------------------------------------|
| Java SE       | Langage de programmation principal                    |
| MySQL         | Systeme de gestion de base de donnees                 |
| JDBC          | Connexion entre Java et MySQL                         |
| DAO           | Separation des requetes SQL du reste de l'application |
| IntelliJ IDEA | Environnement de developpement utilise dans le projet |

### 3. Architecture du système

Le projet est organisé selon une architecture MVC combinée au patron DAO. La couche model contient les classes métier, la couche view contient les interfaces Swing, la couche controller traite les actions utilisateur, la couche dao contient les opérations CRUD, et la couche config contient la connexion JDBC.

#### 3.1 Organisation actuelle du projet

| Package    | Role                                                              | Classes principales                                                                                         |
|------------|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| model      | Représente les entités métier du système                          | Utilisateur, Etudiant, Enseignant, Personnel, Matière, Note                                                 |
| view       | Affiche les interfaces graphiques Swing                           | LoginFrame, AdminView, EtudiantView, EnseignantView, MatièreView, NoteView                                  |
| controller | Traite les actions utilisateur et coordonne les vues avec les DAO | UtilisateurController, EtudiantController, EnseignantController, MatièreController, NoteController          |
| dao        | Gère les opérations CRUD et les requêtes SQL                      | GenericDao, EtudiantDaoImp, EnseignantDaoImp, MatièreDaoImp, NoteDaoImp, PersonnelDaoImp, UtilisateurDaoImp |
| config     | Centralise la connexion JDBC à MySQL                              | Connexion                                                                                                   |

#### 3.2 Flux de données

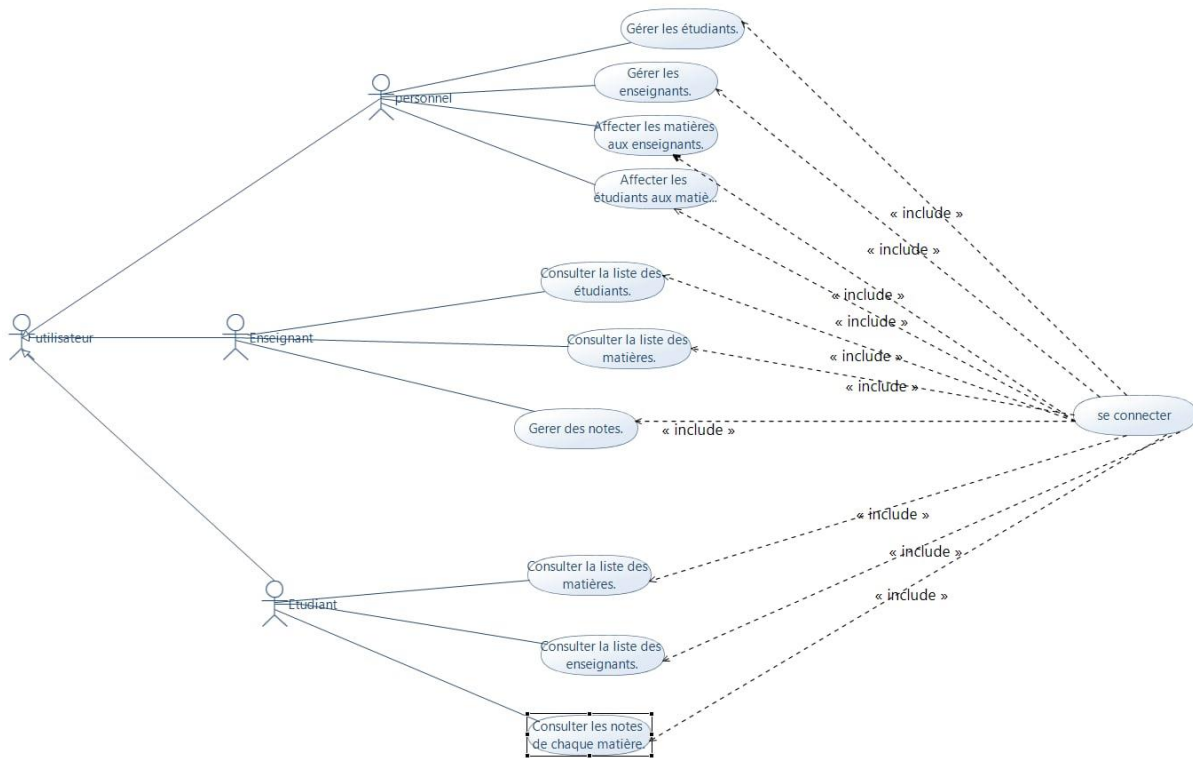
Lorsqu'un utilisateur clique sur un bouton dans une vue Swing, le contrôleur correspondant récupère les données du formulaire, crée ou modifie un objet model, puis appelle le DAO approprié. Le DAO prépare la requête SQL avec PreparedStatement, exécute la requête sur MySQL, puis renvoie le résultat au contrôleur pour mettre à jour la vue.

**Vue Swing -> Contrôleur -> DAO -> JDBC -> MySQL**

La connexion principale utilisée par les DAO est config.Connexion, qui lit les paramètres du fichier configuration.properties : jdbc.driver, jdbc.url, jdbc.user et jdbc.password.

## 4. Conception UML

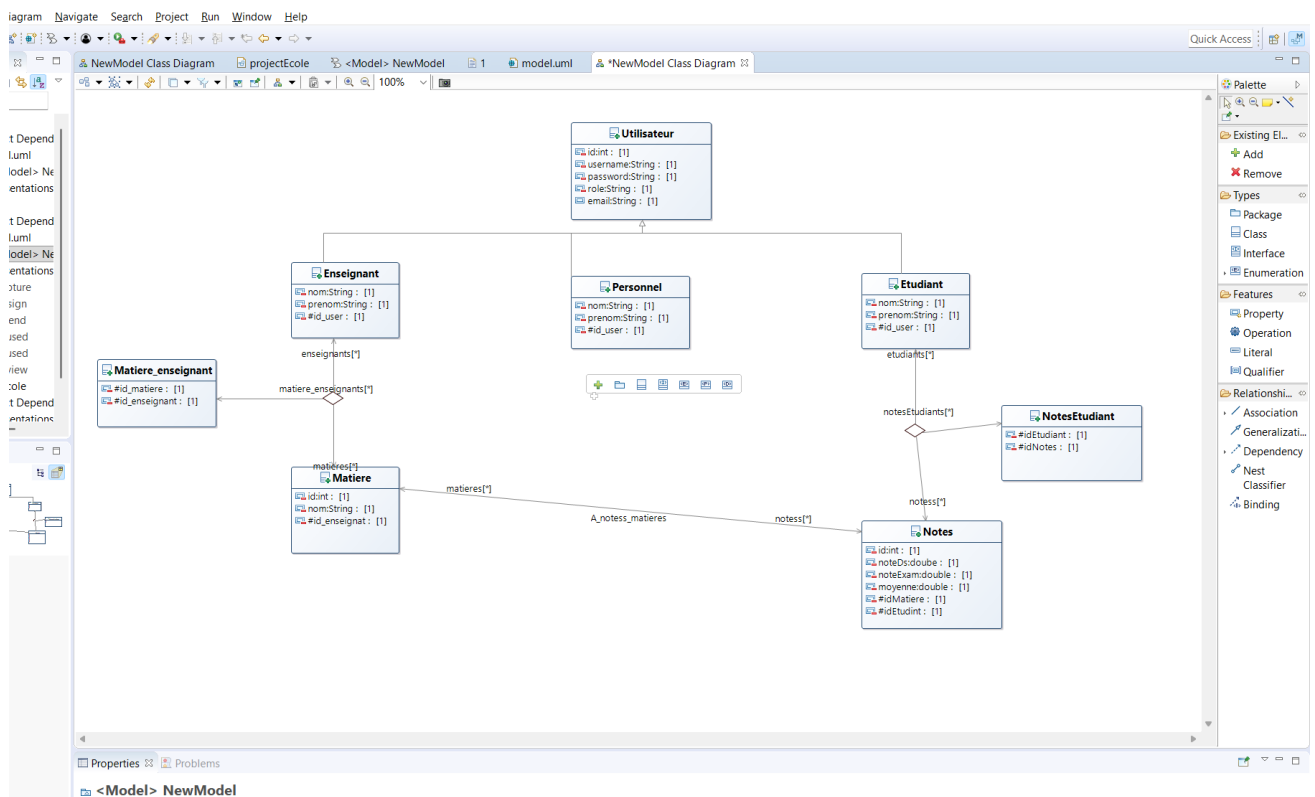
### 4.1 Diagramme des cas d'utilisation



## 4.2 Diagramme de classes

Les relations entre les classes du projet sont les suivantes :

- Etudiant herite de Utilisateur et reutilise id, nom, prenom et email avec super(...)
- Enseignant herite de Utilisateur et ajoute un attribut grade dans le modele Java
- Personnel herite de Utilisateur et utilise aussi le constructeur super(...)
- Note contient une valeur, un Etudiant, une Matiere et un Enseignant ; Matiere contient un id, un nom et un Enseignant
- 



## 5. Base de données

### 5.1 Structure des tables

La base de données définie dans sql/sql.sql contient les tables suivantes : utilisateur, etudiant, enseignant, matiere, enseignant\_matiere, etudiant\_matiere et note. Les noms ci-dessous suivent le script SQL du projet.

**Table : utilisateur**

| Colonne  | Type                                  | Contrainte         | Description           |
|----------|---------------------------------------|--------------------|-----------------------|
| id       | INT                                   | PK, AUTO_INCREMENT | Identifiant unique    |
| username | VARCHAR(50)                           | UNIQUE, NOT NULL   | Nom d'utilisateur     |
| password | VARCHAR(255)                          | NOT NULL           | Mot de passe          |
| role     | ENUM('ADMIN','ENSEIGNANT','ETUDIANT') | NOT NULL           | Rôle de l'utilisateur |

**Table : etudiant**

| Colonne        | Type         | Contrainte            | Description                      |
|----------------|--------------|-----------------------|----------------------------------|
| id             | INT          | PK, AUTO_INCREMENT    | Identifiant unique de l'étudiant |
| nom            | VARCHAR(100) | NOT NULL              | Nom                              |
| prenom         | VARCHAR(100) | NOT NULL              | Prenom                           |
| email          | VARCHAR(150) |                       | Adresse e-mail                   |
| utilisateur_id | INT          | FK -> utilisateur(id) | Compte utilisateur associé       |

**Table : personnel**

| Colonne        | Type         | Contrainte            | Description                      |
|----------------|--------------|-----------------------|----------------------------------|
| id             | INT          | PK, AUTO_INCREMENT    | Identifiant unique de l'étudiant |
| nom            | VARCHAR(100) | NOT NULL              | Nom                              |
| prenom         | VARCHAR(100) | NOT NULL              | Prenom                           |
| email          | VARCHAR(150) |                       | Adresse e-mail                   |
| utilisateur_id | INT          | FK -> utilisateur(id) | Compte utilisateur associé       |

**Table : enseignant**

| Colonne        | Type         | Contrainte            | Description                |
|----------------|--------------|-----------------------|----------------------------|
| id             | INT          | PK, AUTO_INCREMENT    | Identifiant unique         |
| nom            | VARCHAR(100) | NOT NULL              | Nom                        |
| prenom         | VARCHAR(100) | NOT NULL              | Prenom                     |
| email          | VARCHAR(150) |                       | Adresse e-mail             |
| grade          | VARCHAR(100) |                       | Rank (ex:maitre assistant) |
| utilisateur_id | INT          | FK -> utilisateur(id) | Compte utilisateur associe |

**Table : matiere**

| Colonne | Type         | Contrainte         | Description                      |
|---------|--------------|--------------------|----------------------------------|
| id      | INT          | PK, AUTO_INCREMENT | Identifiant unique de la matiere |
| nom     | VARCHAR(100) | NOT NULL           | Nom de la matiere                |

**Table : note**

| Colonne     | Type   | Contrainte                   | Description                            |
|-------------|--------|------------------------------|----------------------------------------|
| id          | INT    | PK, AUTO_INCREMENT           | Identifiant de la note                 |
| etudiant_id | INT    | FK -> etudiant(id), NOT NULL | Etudiant concerne                      |
| matiere_id  | INT    | FK -> matiere(id), NOT NULL  | Matiere concerne                       |
| note_ds     | DOUBLE |                              | Note du devoir surveille               |
| note_exam   | DOUBLE |                              | Note d'examen                          |
| moyenne     | DOUBLE |                              | Note finale utilisee par le DAO actuel |

**Table : etudiant\_matiere**

| Colonne     | Type | Contrainte             | Description        |
|-------------|------|------------------------|--------------------|
| etudiant_id | INT  | PK, FK -> etudiant(id) | Reference etudiant |
| matiere_id  | INT  | PK, FK -> matiere(id)  | Reference matiere  |

**Table : enseignant\_matiere**

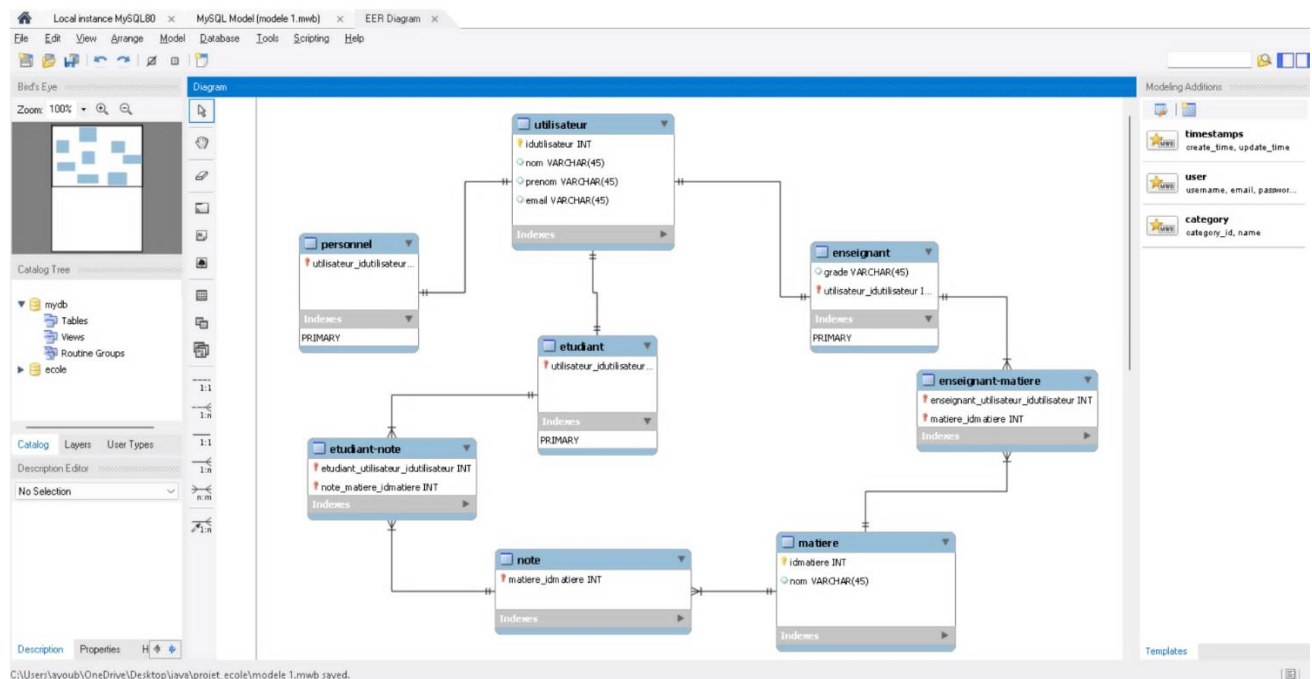
| Colonne       | Type | Contrainte               | Description          |
|---------------|------|--------------------------|----------------------|
| enseignant_id | INT  | PK, FK -> enseignant(id) | Reference enseignant |
| matiere_id    | INT  | PK, FK -> matiere(id)    | Reference matiere    |

## 5.2 Schéma relationnel

Le schema relationnel reel du script SQL est le suivant :

### Schema relationnel - school\_db

```
utilisateur (id, username, password, role)
etudiant (id, nom, prenom, email, #utilisateur_id)
enseignant (id, nom, prenom, email, grade, #utilisateur_id)
personnel (id, nom, prenom, email, #utilisateur_id)
matiere (id, nom)
note (id, #etudiant_id, #matiere_id, note_ds, note_exam, moyenne)
etudiant_matiere (#etudiant_id, #matiere_id)
enseignant_matiere (#enseignant_id, #matiere_id)
```



## 6. Structure du projet Java

Le projet est organise selon une architecture MVC avec des packages bien separees et une couche DAO dediee a la persistance des donnees.

| Package / Classe       | Role                                                                     |
|------------------------|--------------------------------------------------------------------------|
| model/Utilisateur.java | Classe mere : id, nom, prenom, email                                     |
| model/Etudiant.java    | Herite de Utilisateur                                                    |
| model/Enseignant.java  | Herite de Utilisateur et ajoute grade                                    |
| model/Personnel.java   | Herite de Utilisateur avec constructeur super(...)                       |
| model/Matiere.java     | id, nom, enseignant                                                      |
| model/Note.java        | id, valeur, etudiant, matiere, enseignant                                |
| view/*.java            | Interfaces graphiques Swing : connexion, tableaux de bord et formulaires |
| controller/*.java      | Traitement des actions utilisateur et appel des DAO                      |
| dao/GenericDao.java    | Interface generique : add, findById, findAll, update, delete             |
| dao/*DaoImp.java       | Implementations JDBC des operations CRUD                                 |
| config/Connexion.java  | Connexion MySQL via configuration.properties                             |

## 7. Description des interfaces graphiques

### 7.1 Interface de connexion

La première interface affichée est la fenêtre de connexion. Elle contient un champ pour le nom d'utilisateur, un champ pour le mot de passe et un bouton Se connecter. Après validation, le contrôleur vérifie les informations via UtilisateurDaoImp et redirige l'utilisateur selon son rôle.

### 7.2 Interface de gestion des étudiants

L'interface de gestion des étudiants affiche les données dans une JTable et propose les boutons Ajouter, Modifier, Supprimer, Enregistrer et Fermer. Les actions sont transmises au contrôleur, qui utilise EtudiantDaoImp pour exécuter les opérations CRUD.

| Methode        | Role                                     |
|----------------|------------------------------------------|
| add(entity)    | Inserer une nouvelle ligne dans la table |
| findById(id)   | Chercher une ligne par son identifiant   |
| findAll()      | Lister toutes les lignes de la table     |
| update(entity) | Modifier une ligne existante avec son id |
| delete(id)     | Supprimer une ligne par son identifiant  |

### 7.3 Interface de gestion des enseignants

L'interface enseignants permet la gestion des enseignants avec les champs id, nom, prenom, email et specialite. Le contrôleur appelle EnseignantDaoImp pour ajouter, rechercher, modifier, lister ou supprimer un enseignant.

### 7.4 Interface de gestion des notes

L'interface des notes permet d'associer un étudiant, une matière et une note finale. Les données sont envoyées au contrôleur, puis sauvegardées dans la table note via NoteDaoImp.

### 7.5 Interface tableau de bord - Enseignant

L'enseignant consulte ses matières, les étudiants associés et les notes. Les informations sont chargées depuis les tables enseignant, matiere, enseignant\_matiere, etudiant\_matiere et note.

### 7.6 Interface tableau de bord - Etudiant

L'étudiant consulte ses coordonnées, ses matières et ses notes. Les vues affichent les informations en lecture seule, tandis que les DAO récupèrent les données correspondantes depuis MySQL.

## 8. Fonctionnalités par entité

### 8.1 Administrateur / Personnel

Le personnel administratif est modelise par la classe Personnel, qui herite maintenant correctement de Utilisateur. Dans la base actuelle, il n'existe pas encore de table personnel separee ; le DAO utilise la table utilisateur avec le role ADMIN.

| Fonctionnalite         | Detail                                                            |
|------------------------|-------------------------------------------------------------------|
| Gerer les utilisateurs | Creer, modifier, lister et supprimer les comptes dans utilisateur |
| Gerer les etudiants    | CRUD sur la table etudiant via EtudiantDaoImp                     |
| Gerer les enseignants  | CRUD sur la table enseignant via EnseignantDaoImp                 |
| Gerer les matieres     | CRUD sur la table matiere via MatiereDaoImp                       |
| Gerer les notes        | CRUD sur la table note via NoteDaoImp                             |

## 8.2 Enseignant

| Fonctionnalite         | Detail                                                       |
|------------------------|--------------------------------------------------------------|
| Consulter son profil   | Donnees heritees de Utilisateur : id, nom, prenom, email     |
| Consulter ses matieres | Association possible avec enseignant_matiere                 |
| Gerer les notes        | Ajout, modification, recherche et suppression via NoteDaoImp |

## 8.3 Étudiant

| Fonctionnalite         | Detail                                                   |
|------------------------|----------------------------------------------------------|
| Consulter son profil   | Donnees heritees de Utilisateur : id, nom, prenom, email |
| Consulter ses matieres | Association possible avec etudiant_matiere               |
| Consulter ses notes    | Lecture des notes liees a etudiant_id dans la table note |

## 8.4 Extrait de code - GenericDao.java

```
public interface GenericDao<T> {
```

```
    void add(T entity);  
    T findById(int id);  
    List<T> findAll();  
    void update(T entity);  
    void delete(int id);  
}
```

Cette interface est implementee par les classes EtudiantDaoImp, EnseignantDaoImp, MatiereDaoImp, NoteDaoImp, PersonnelDaoImp et UtilisateurDaoImp.

## 9. Conclusion

Ce projet met en place les bases d'une application Java de gestion d'école : des classes modele, une connexion JDBC a MySQL et des DAO pour isoler les requetes SQL.

Grace a cette structure, le code est plus lisible : les modeles representent les donnees, les DAO s'occupent de la base, et la connexion est centralisee dans un seul package.

La version actuelle du systeme couvre les operations CRUD principales pour les etudiants, enseignants, matieres, notes, utilisateurs et personnel, avec une organisation MVC permettant de relier les vues Swing, les controleurs, les DAO et la base MySQL.

Les ameliorations futures prioritaires sont l'harmonisation entre modeles et tables SQL, l'ajout d'interfaces graphiques, la creation de controleurs, la securisation des mots de passe et le calcul complet des moyennes.